# Tutorial 6

Teaching Team of STA314

## Roadmap

In this tutorial, we will:

1. Investigate **complete separation** in logistic regression and see how regularization helps.
2. Compare **logistic regression, LDA, and QDA** on the **Smarket** data.
3. Compare **multinomial logistic regression, LDA, and QDA** on the **iris** data, including their decision regions.
4. Discuss how model assumptions, flexibility, and test accuracy relate to model choice.

### Setup

Run the following installation command **only if** any of the packages are missing:

```
install.packages(c("ISLR2", "MASS", "glmnet", "nnet", "ggplot2"))
```

```
library(ISLR2)      # Smarket dataset
library(MASS)       # lda() and qda()
library(glmnet)     # Regularized logistic regression
library(nnet)       # Multinomial logistic regression
library(ggplot2)    # Plots

set.seed(0)
```

## 1. Warm-up: Complete separation in logistic regression

For binary logistic regression, we model

$$P(Y = 1 \mid \mathbf{X} = \mathbf{x}) = \frac{\exp(\beta_0 + \mathbf{x}^T\boldsymbol{\beta})}{1 + \exp(\beta_0 + \mathbf{x}^T\boldsymbol{\beta})}.$$

The unregularized maximum likelihood estimator maximizes the log-likelihood. If the two classes can be **completely separated**, increasing the magnitude of a separating coefficient vector drives the fitted probabilities toward 0 and 1, without attaining a finite maximum likelihood estimate.

Below we create a toy **separable** dataset and show that unregularized fitting can produce huge coefficients or warnings, while regularization yields a stable, finite solution.

### Simulate separable data

```
set.seed(0)

x0 <- rnorm(10, mean = 0, sd = 1)
x1 <- rnorm(10, mean = 5, sd = 1)

separation_data <- data.frame(
```

```
  x = c(x0, x1),
  z = rnorm(20),
  y = c(rep(0, 10), rep(1, 10))
)

# Check whether the two observed groups are separated
max(x0) < min(x1)
```

```
## [1] TRUE
```

```
head(separation_data)
```

```
##            x           z y
## 1  1.2629543 -0.22426789 0
## 2 -0.3262334  0.37739565 0
## 3  1.3297993  0.13333636 0
## 4  1.2724293  0.80418951 0
## 5  0.4146414 -0.05710677 0
## 6 -1.5399500  0.50360797 0
```

### Fit ordinary logistic regression

```
# Warnings about fitted probabilities of 0 or 1 may appear.
fit_sep <- glm(y ~ x + z, data = separation_data, family = binomial())
```

```
## Warning: glm.fit: algorithm did not converge
```

```
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
```

```
summary(fit_sep)
```

```
##
## Call:
## glm(formula = y ~ x + z, family = binomial(), data = separation_data)
##
## Coefficients:
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept)    -92.57  137457.61  -0.001    0.999
## x               29.47   42943.45   0.001    0.999
## z              -16.08   85866.65   0.000    1.000
##
## (Dispersion parameter for binomial family taken to be 1)
##
##     Null deviance: 2.7726e+01  on 19  degrees of freedom
## Residual deviance: 6.7028e-10  on 17  degrees of freedom
## AIC: 6
##
## Number of Fisher Scoring iterations: 25
```

```
coef(fit_sep)
```

```
## (Intercept)           x           z
##   -92.57119    29.47318   -16.08139
```

```
x_grid <- data.frame(
  x = seq(min(separation_data$x) - 1,
          max(separation_data$x) + 1,
```

```
          length.out = 400),
  z = 0
)

x_grid$p <- predict(fit_sep, newdata = x_grid, type = "response")

ggplot(separation_data, aes(x = x, y = y)) +
  geom_point(aes(color = factor(y)), size = 2.5) +
  geom_line(data = x_grid, aes(y = p), color = "darkgreen", linewidth = 1) +
  scale_color_manual(values = c("0" = "steelblue", "1" = "firebrick"),
                     name = "Class") +
  labs(title = "Logistic regression with separated classes",
       x = "x", y = "Observed class / fitted probability") +
  theme_minimal()
```




**Fit ridge-regularized logistic regression**

`glmnet()` minimizes a penalized loss. With `alpha = 0`, it uses an L2 (ridge) penalty. We specify a fixed positive `lambda` for illustration rather than choosing it by cross-validation.

```
X_sep <- as.matrix(separation_data[, c("x", "z")])
y_sep <- separation_data$y

ridge_sep <- glmnet(X_sep, y_sep,
                    family = "binomial", alpha = 0, lambda = 0.1)
coef(ridge_sep)
```

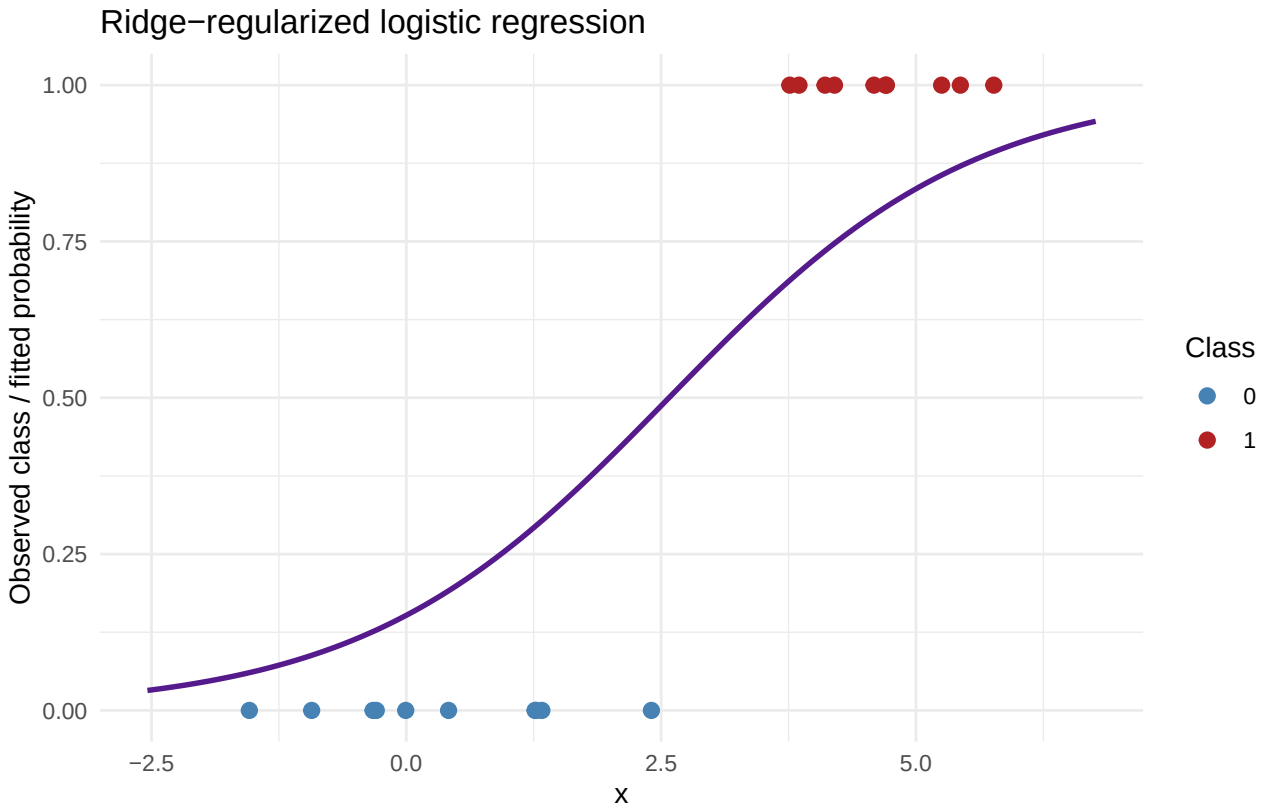
```
## 3 x 1 sparse Matrix of class "dgCMatrix"
##                       s0
```

```
## (Intercept) -1.71795125
## x            0.66664769
## z           -0.02086307
```

```r
x_new <- cbind(x = x_grid$x, z = 0)

x_grid$p_ridge <- as.numeric(
  predict(ridge_sep, newx = x_new,
          type = "response", s = 0.1)
)
```

```r
ggplot(separation_data, aes(x = x, y = y)) +
  geom_point(aes(color = factor(y)), size = 2.5) +
  geom_line(data = x_grid, aes(y = p_ridge),
            color = "purple4", linewidth = 1) +
  scale_color_manual(values = c("0" = "steelblue", "1" = "firebrick"),
                     name = "Class") +
  labs(title = "Ridge-regularized logistic regression",
       x = "x", y = "Observed class / fitted probability") +
  theme_minimal()
```




**Key point:** Under complete separation, an unregularized logistic regression fit may not have finite MLEs; regularization yields finite coefficients for a positive penalty.

## 2. A reusable function for decision regions

The function below fits no model itself. Given a **fitted classifier** and two predictors, it creates a grid, predicts the class at each grid point, and colors the resulting decision regions. We will use it for both datasets.

```r
plot_decision_regions <- function(model, data, features, response, title,
                                  grid_length = 180) {
  stopifnot(length(features) == 2)

  x_name <- features[1]
  y_name <- features[2]

  x_range <- range(data[[x_name]]) + c(-1, 1)
  y_range <- range(data[[y_name]]) + c(-1, 1)

  grid <- expand.grid(
    seq(x_range[1], x_range[2], length.out = grid_length),
    seq(y_range[1], y_range[2], length.out = grid_length)
  )
  names(grid) <- features

  # predict() returns class labels for glmnet-independent classifiers
  grid$predicted_class <- factor(predict(model, newdata = grid, type = "class"))
  plot_data <- data
  plot_data$observed_class <- factor(data[[response]])

  ggplot() +
    geom_raster(data = grid,
                aes(x = .data[[x_name]], y = .data[[y_name]],
                    fill = predicted_class), alpha = 0.3) +
    geom_point(data = plot_data,
               aes(x = .data[[x_name]], y = .data[[y_name]],
                   color = observed_class), size = 1.6, alpha = 0.8) +
    labs(title = title, x = x_name, y = y_name,
         fill = "Predicted", color = "Observed") +
    coord_cartesian(expand = FALSE) +
    theme_minimal()
}
```

**Note:** The helper is intended for the `glm`-independent `MASS::lda`, `MASS::qda` and `nnet::multinom` classifiers below, plus the logistic regression wrapper defined in the Smarket section. Its use of `type = "class"` will not work directly with a base R `glm` object.

## 3. Binary classification: Smarket

The `Smarket` dataset has 1,250 observations of daily S&P 500 returns from 2001 to 2005. `Lag1` through `Lag5` are percentage returns on the previous five trading days, `Volume` is trading volume, and `Direction` indicates whether the market moved **Up** or **Down**.

```r
data(Smarket)
head(Smarket)
```

```
##   Year   Lag1   Lag2   Lag3   Lag4   Lag5 Volume  Today Direction
## 1 2001  0.381 -0.192 -2.624 -1.055  5.010 1.1913  0.959        Up
## 2 2001  0.959  0.381 -0.192 -2.624 -1.055 1.2965  1.032        Up
## 3 2001  1.032  0.959  0.381 -0.192 -2.624 1.4112 -0.623      Down
## 4 2001 -0.623  1.032  0.959  0.381 -0.192 1.2760  0.614        Up
## 5 2001  0.614 -0.623  1.032  0.959  0.381 1.2057  0.213        Up
## 6 2001  0.213  0.614 -0.623  1.032  0.959 1.3491  1.392        Up
```

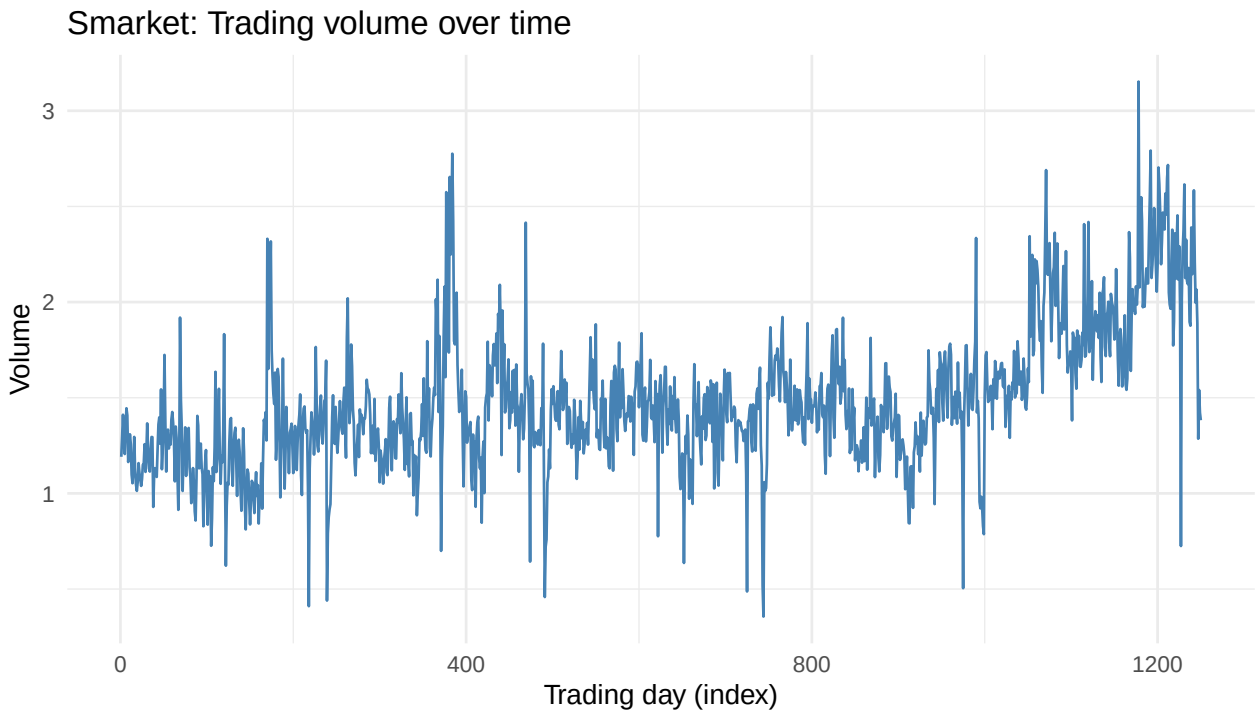
```r
str(Smarket)
```

```
## 'data.frame':    1250 obs. of  9 variables:
##  $ Year     : num  2001 2001 2001 2001 2001 ...
##  $ Lag1     : num  0.381 0.959 1.032 -0.623 0.614 ...
##  $ Lag2     : num  -0.192 0.381 0.959 1.032 -0.623 ...
##  $ Lag3     : num  -2.624 -0.192 0.381 0.959 1.032 ...
##  $ Lag4     : num  -1.055 -2.624 -0.192 0.381 0.959 ...
##  $ Lag5     : num  5.01 -1.055 -2.624 -0.192 0.381 ...
##  $ Volume   : num  1.19 1.3 1.41 1.28 1.21 ...
##  $ Today    : num  0.959 1.032 -0.623 0.614 0.213 ...
##  $ Direction: Factor w/ 2 levels "Down","Up": 2 2 1 2 2 2 1 2 2 2 ...
```

```r
cor(Smarket[, sapply(Smarket, is.numeric)])
```

```
##              Year         Lag1         Lag2         Lag3         Lag4
## Year   1.00000000  0.029699649  0.030596422  0.033194581  0.035688718
## Lag1   0.02969965  1.000000000 -0.026294328 -0.010803402 -0.002985911
## Lag2   0.03059642 -0.026294328  1.000000000 -0.025896670 -0.010853533
## Lag3   0.03319458 -0.010803402 -0.025896670  1.000000000 -0.024051036
## Lag4   0.03568872 -0.002985911 -0.010853533 -0.024051036  1.000000000
## Lag5   0.02978799 -0.005674606 -0.003557949 -0.018808338 -0.027083641
## Volume 0.53900647  0.040909908 -0.043383215 -0.041823686 -0.048414246
## Today  0.03009523 -0.026155045 -0.010250033 -0.002447647 -0.006899527
##                Lag5      Volume        Today
## Year    0.029787995  0.53900647  0.030095229
## Lag1   -0.005674606  0.04090991 -0.026155045
## Lag2   -0.003557949 -0.04338321 -0.010250033
## Lag3   -0.018808338 -0.04182369 -0.002447647
## Lag4   -0.027083641 -0.04841425 -0.006899527
## Lag5    1.000000000 -0.02200231 -0.034860083
## Volume -0.022002315  1.00000000  0.014591823
## Today  -0.034860083  0.01459182  1.000000000
```

```r
ggplot(Smarket, aes(x = seq_along(Volume), y = Volume)) +
  geom_line(color = "steelblue") +
  labs(title = "Smarket: Trading volume over time",
       x = "Trading day (index)", y = "Volume") +
  theme_minimal()
```



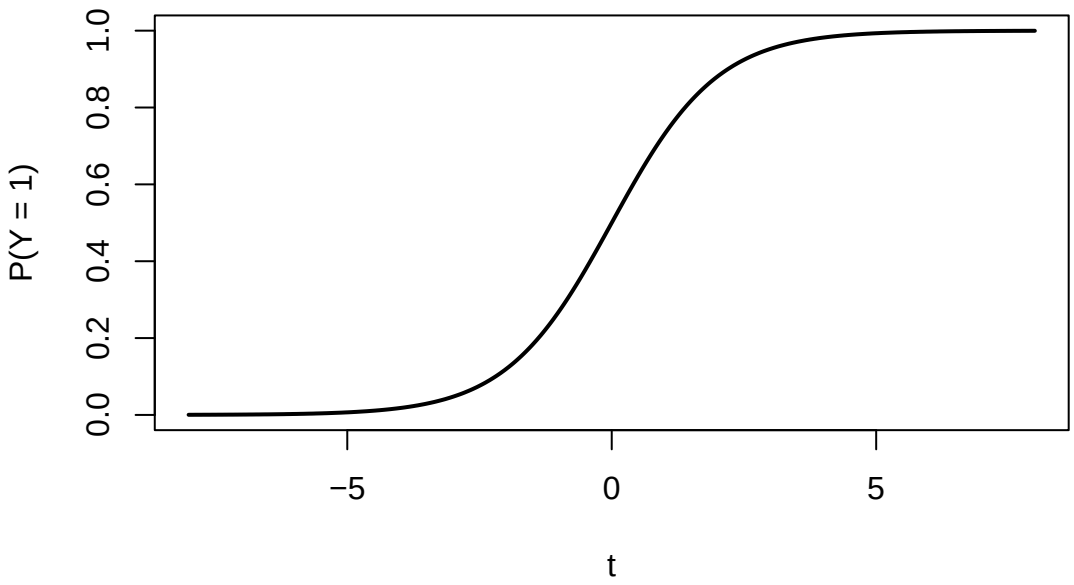

### 3.1 Logistic regression using all lag variables and Volume

Binary logistic regression models the posterior probability **directly**:

$$\log\left(\frac{P(Y=\text{Up}\mid\mathbf{x})}{1-P(Y=\text{Up}\mid\mathbf{x})}\right)=\beta_0+\mathbf{x}^T\boldsymbol{\beta}.$$

With a 0.5 probability cutoff, its decision boundary is linear in the original predictors.

```
t <- seq(-8, 8, length.out = 400)
plot(t, plogis(t), type = "l", lwd = 2,
     main = "Sigmoid function", xlab = "t", ylab = "P(Y = 1)")
```




```
logit_all <- glm(Direction ~ Lag1 + Lag2 + Lag3 + Lag4 + Lag5 + Volume,
                 data = Smarket, family = binomial())
summary(logit_all)
```

```
##
## Call:
## glm(formula = Direction ~ Lag1 + Lag2 + Lag3 + Lag4 + Lag5 +
##     Volume, family = binomial(), data = Smarket)
##
## Coefficients:
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept) -0.126000   0.240736  -0.523    0.601
## Lag1        -0.073074   0.050167  -1.457    0.145
## Lag2        -0.042301   0.050086  -0.845    0.398
## Lag3         0.011085   0.049939   0.222    0.824
## Lag4         0.009359   0.049974   0.187    0.851
## Lag5         0.010313   0.049511   0.208    0.835
## Volume       0.135441   0.158360   0.855    0.392
##
## (Dispersion parameter for binomial family taken to be 1)
##
##     Null deviance: 1731.2  on 1249  degrees of freedom
## Residual deviance: 1727.6  on 1243  degrees of freedom
## AIC: 1741.6
##
## Number of Fisher Scoring iterations: 3
```

```
prob_all <- predict(logit_all, type = "response")
head(prob_all, 10)
```

```
##         1         2         3         4         5         6         7         8
## 0.5070841 0.4814679 0.4811388 0.5152224 0.5107812 0.5069565 0.4926509 0.5092292
##         9        10
## 0.5176135 0.4888378
```

```r
pred_all <- factor(ifelse(prob_all > 0.5, "Up", "Down"),
                   levels = levels(Smarket$Direction))

# Rows: predictions; columns: true classes
table(Predicted = pred_all, Truth = Smarket$Direction)
```

```
##          Truth
## Predicted Down  Up
##      Down  145 141
##      Up    457 507
```

```r
mean(pred_all == Smarket$Direction)  # Training accuracy
```

```
## [1] 0.5216
```

### 3.2 Evaluate on a held-out year

Train on observations from **2001–2004** and test on **2005**. This chronological split avoids training on observations that occur after the test observations.

```r
train <- subset(Smarket, Year < 2005)
test  <- subset(Smarket, Year == 2005)

nrow(train)
```

```
## [1] 998
```

```r
nrow(test)
```

```
## [1] 252
```

```r
logit_all_train <- glm(Direction ~ Lag1 + Lag2 + Lag3 + Lag4 + Lag5 + Volume,
                       data = train, family = binomial())

prob_test <- predict(logit_all_train, newdata = test, type = "response")
pred_test <- factor(ifelse(prob_test > 0.5, "Up", "Down"),
                    levels = levels(Smarket$Direction))

table(Predicted = pred_test, Truth = test$Direction)
```

```
##          Truth
## Predicted Down Up
##      Down   77 97
##      Up     34 44
```

```r
mean(pred_test == test$Direction)  # Test accuracy
```

```
## [1] 0.4801587
```

```r
# Accuracy from predicting the most frequent class in the training set
majority_class <- names(which.max(table(train$Direction)))
majority_class
```

```
## [1] "Up"
```

```r
mean(test$Direction == majority_class)
```

```
## [1] 0.5595238
```

### 3.3 Refit using only Lag1 and Lag2

Using just two predictors lets us visualize the decision boundaries. For a fair comparison, all three models use **exactly the same training and test data**.

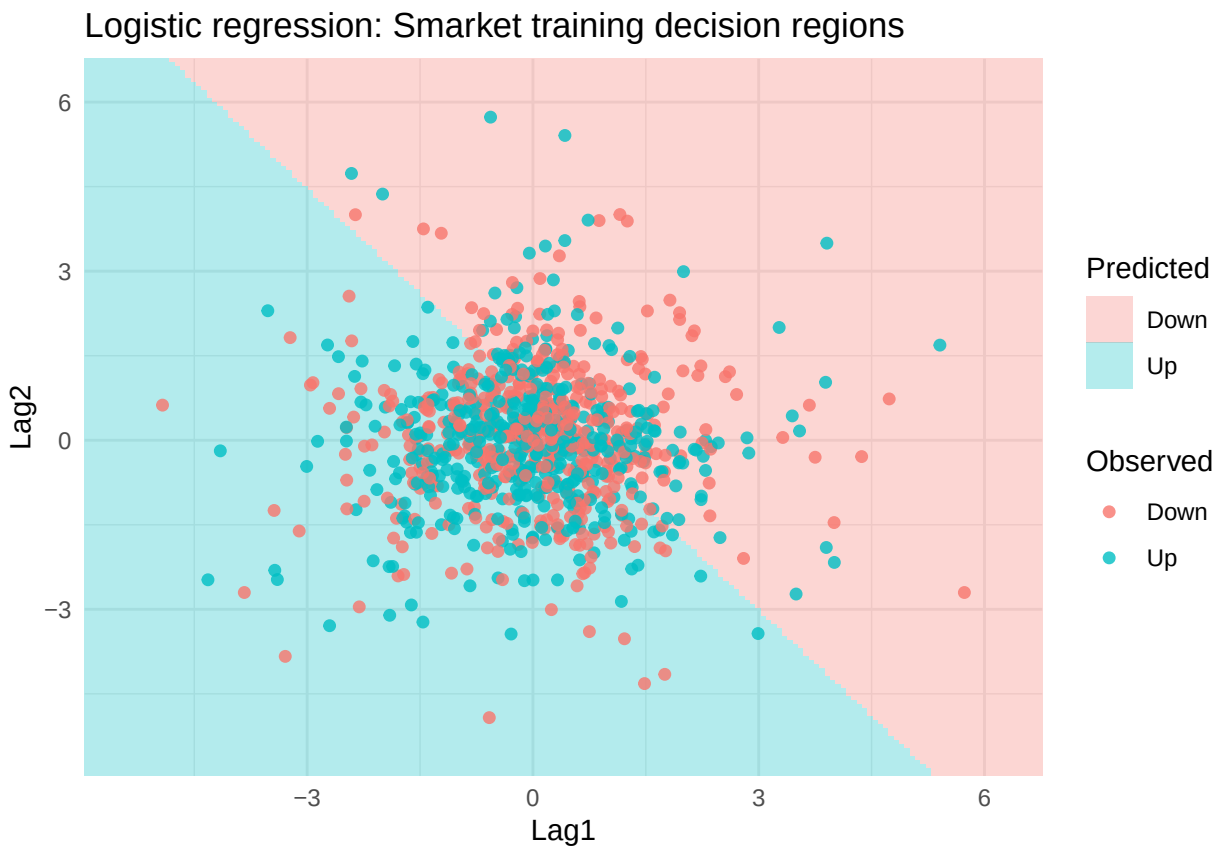
```
features <- c("Lag1", "Lag2")

# Fit an ordinary (unpenalized) logistic regression, equivalent to glm(..., binomial).
logit_two <- glm(Direction ~ Lag1 + Lag2,
                 data = train, family = binomial())

# Wrapper providing class predictions for the plotting helper
logit_class <- list(fit = logit_two)
class(logit_class) <- "logit_class"

predict.logit_class <- function(object, newdata, type = "class", ...) {
  p <- predict(object$fit, newdata = newdata, type = "response")
  factor(ifelse(p > 0.5, "Up", "Down"), levels = c("Down", "Up"))
}

plot_decision_regions(logit_class, train, features, "Direction",
                      "Logistic regression: Smarket training decision regions")
```




```
pred_logit <- predict(logit_class, newdata = test)
table(Predicted = pred_logit, Truth = test$Direction)
```

```
##           Truth
## Predicted Down  Up
```

```
##      Down   35  35
##      Up     76 106
```

```
mean(pred_logit == test$Direction)
```

```
## [1] 0.5595238
```

### 3.4 Linear discriminant analysis (LDA)

LDA assumes the predictor distributions within the classes are multivariate Gaussian with a **shared covariance matrix**:

$$\mathbf{X} \mid Y = k \sim N_p(\boldsymbol{\mu}_k, \boldsymbol{\Sigma}).$$

Its discriminant score is

$$\delta_k(\mathbf{x}) = \mathbf{x}^T \boldsymbol{\Sigma}^{-1} \boldsymbol{\mu}_k - \frac{1}{2} \boldsymbol{\mu}_k^T \boldsymbol{\Sigma}^{-1} \boldsymbol{\mu}_k + \log \pi_k.$$

```
lda_fit <- lda(Direction ~ Lag1 + Lag2, data = train)
lda_fit
```

```
## Call:
## lda(Direction ~ Lag1 + Lag2, data = train)
##
## Prior probabilities of groups:
##     Down       Up
## 0.491984 0.508016
##
## Group means:
##             Lag1        Lag2
## Down  0.04279022  0.03389409
## Up   -0.03954635 -0.03132544
##
## Coefficients of linear discriminants:
##             LD1
## Lag1 -0.6420190
## Lag2 -0.5135293
```

```
# MASS::predict() returns a list; $class contains class labels.
pred_lda <- predict(lda_fit, newdata = test)$class

table(Predicted = pred_lda, Truth = test$Direction)
```

```
##          Truth
## Predicted Down  Up
##      Down   35  35
##      Up     76 106
```

```
mean(pred_lda == test$Direction)
```

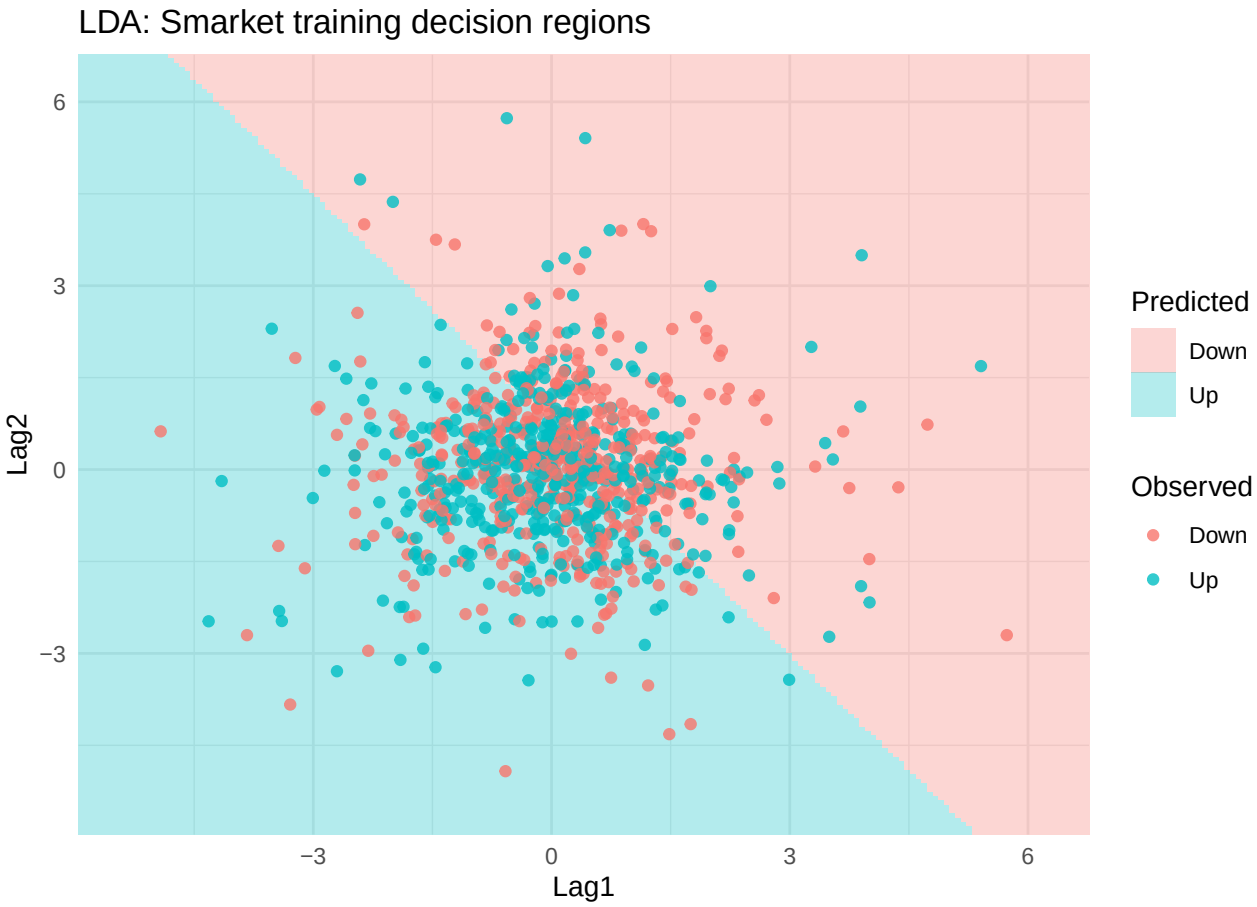
```
## [1] 0.5595238
```

For plotting, wrap LDA's output in a classifier that returns labels:

```
lda_class <- list(fit = lda_fit)
class(lda_class) <- "lda_class"
```

```
predict.lda_class <- function(object, newdata, type = "class", ...) {
  predict(object$fit, newdata = newdata)$class
}

plot_decision_regions(lda_class, train, features, "Direction",
                      "LDA: Smarket training decision regions")
```




### 3.5 Quadratic discriminant analysis (QDA)

QDA assumes class-conditional Gaussians but allows the covariance matrix to **differ by class**:

$$\mathbf{X} \mid Y = k \sim N_p(\boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k).$$

The discriminant score contains a class-specific quadratic term:

$$\delta_k(\mathbf{x}) = -\frac{1}{2}\log|\boldsymbol{\Sigma}_k| - \frac{1}{2}(\mathbf{x}-\boldsymbol{\mu}_k)^T\boldsymbol{\Sigma}_k^{-1}(\mathbf{x}-\boldsymbol{\mu}_k) + \log\pi_k.$$

```
qda_fit <- qda(Direction ~ Lag1 + Lag2, data = train)
qda_fit
```

```
## Call:
## qda(Direction ~ Lag1 + Lag2, data = train)
##
```

```
## Prior probabilities of groups:
##     Down       Up
## 0.491984 0.508016
##
## Group means:
##             Lag1        Lag2
## Down  0.04279022  0.03389409
## Up   -0.03954635 -0.03132544
```

```
pred_qda <- predict(qda_fit, newdata = test)$class

table(Predicted = pred_qda, Truth = test$Direction)
```

```
##          Truth
## Predicted Down  Up
##      Down   30  20
##      Up     81 121
```

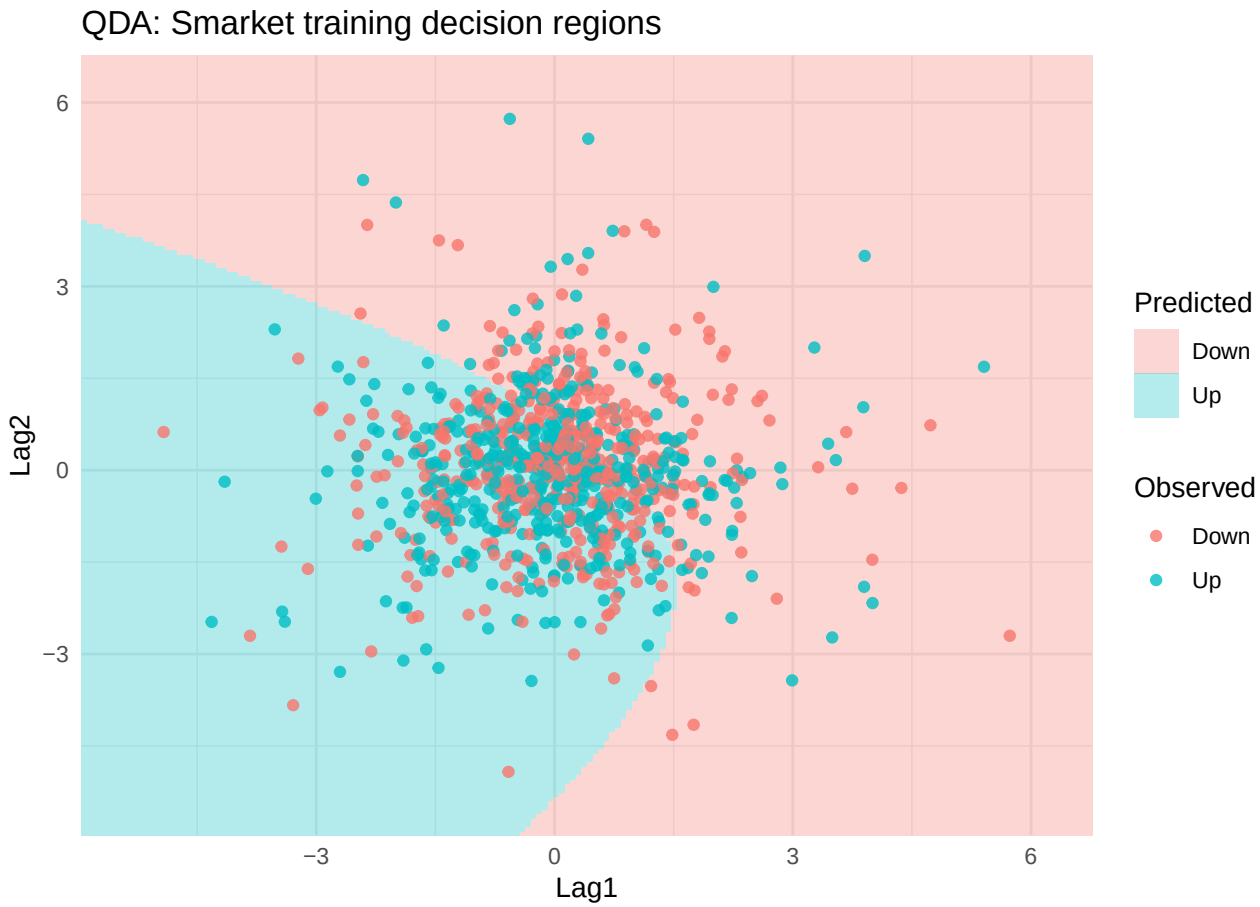
```
mean(pred_qda == test$Direction)
```

```
## [1] 0.5992063
```

```
qda_class <- list(fit = qda_fit)
class(qda_class) <- "qda_class"

predict.qda_class <- function(object, newdata, type = "class", ...) {
  predict(object$fit, newdata = newdata)$class
}

plot_decision_regions(qda_class, train, features, "Direction",
                      "QDA: Smarket training decision regions")
```




### 3.6 Compare Smarket test accuracy

```
results_smarket <- data.frame(
  Model = c("Logistic regression", "LDA", "QDA"),
  Test_Accuracy = c(
    mean(pred_logit == test$Direction),
    mean(pred_lda == test$Direction),
    mean(pred_qda == test$Direction)
  )
)
results_smarket[order(-results_smarket$Test_Accuracy), ]
```

```
##                 Model Test_Accuracy
## 3                 QDA     0.5992063
## 1 Logistic regression     0.5595238
## 2                 LDA     0.5595238
```

## 4. Multiclass classification: Iris

The built-in `iris` dataset contains **three species**, with **50 observations per species**. To visualize decision regions in two dimensions, we use **petal length** and **petal width** only.

```
data(iris)
head(iris)
```

```
##   Sepal.Length Sepal.Width Petal.Length Petal.Width Species
```

```
## 1          5.1         3.5          1.4         0.2  setosa
## 2          4.9         3.0          1.4         0.2  setosa
## 3          4.7         3.2          1.3         0.2  setosa
## 4          4.6         3.1          1.5         0.2  setosa
## 5          5.0         3.6          1.4         0.2  setosa
## 6          5.4         3.9          1.7         0.4  setosa
```

```r
table(iris$Species)
```

```
##
##     setosa versicolor  virginica
##         50         50         50
```

```r
iris_two <- iris[, c("Petal.Length", "Petal.Width", "Species")]

# All three models are fit and displayed on the full iris dataset to
# reproduce the original tutorial's illustrative decision-region plots.
# These are NOT held-out test evaluations.
```

### 4.1 Multinomial logistic regression (softmax)

For $K$ classes, multinomial logistic regression models the class probabilities directly:

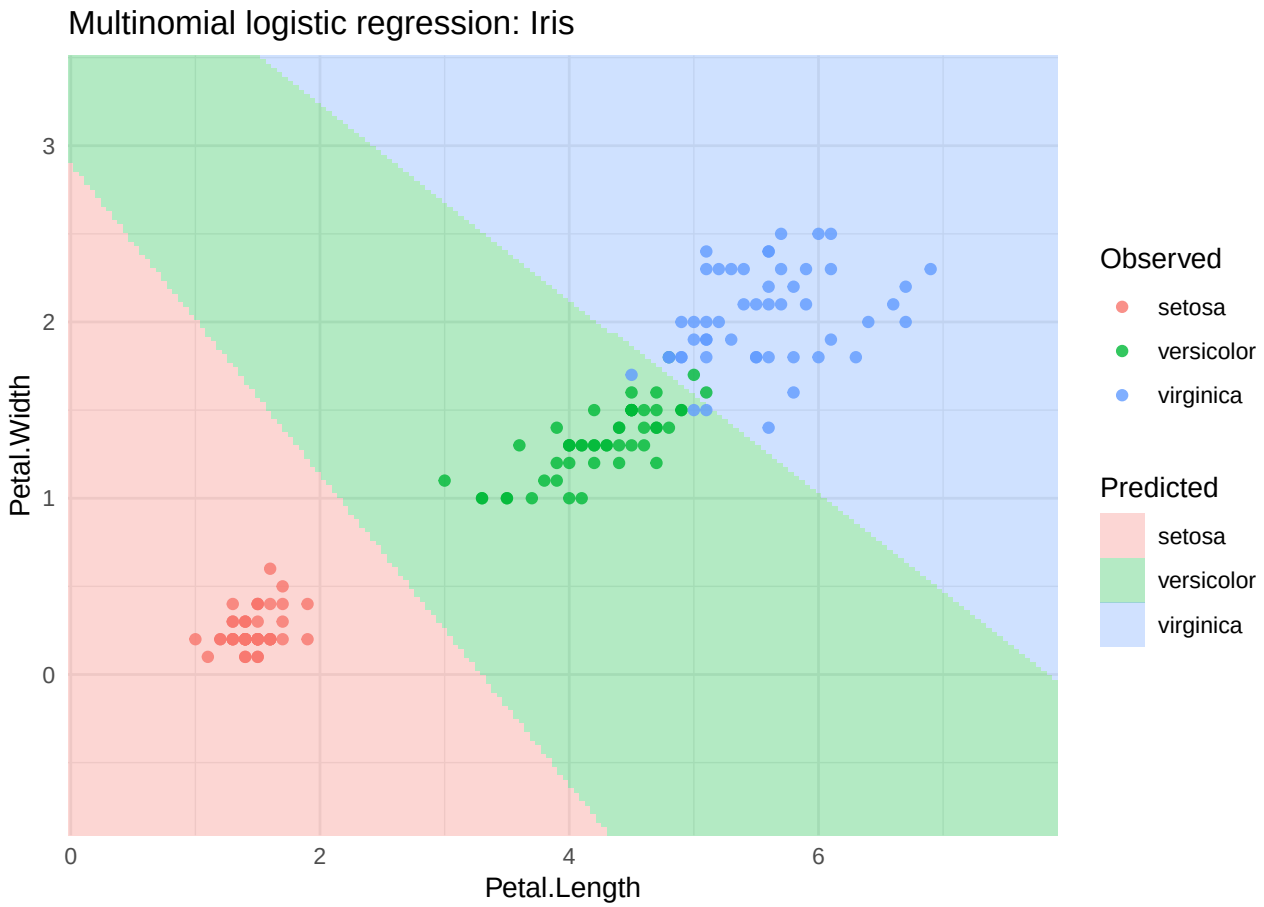
$$P(Y = k \mid \mathbf{x}) = \frac{\exp(\beta_{k0} + \mathbf{x}^T \boldsymbol{\beta}_k)}{\sum_{j=1}^{K} \exp(\beta_{j0} + \mathbf{x}^T \boldsymbol{\beta}_j)}.$$

One class can be used as the reference class for identifiability. The **pairwise decision boundaries** are linear in the original predictors.

```r
multinom_fit <- multinom(Species ~ Petal.Length + Petal.Width,
                         data = iris_two, trace = FALSE)
summary(multinom_fit)
```

```
## Call:
## multinom(formula = Species ~ Petal.Length + Petal.Width, data = iris_two,
##     trace = FALSE)
##
## Coefficients:
##            (Intercept) Petal.Length Petal.Width
## versicolor   -22.79944      6.92122    7.878496
## virginica    -67.82521     12.64721   18.261016
##
## Std. Errors:
##            (Intercept) Petal.Length Petal.Width
## versicolor     44.3859     37.58715    81.00888
## virginica      46.3939     37.65702    81.09482
##
## Residual Deviance: 20.57901
## AIC: 32.57901
```

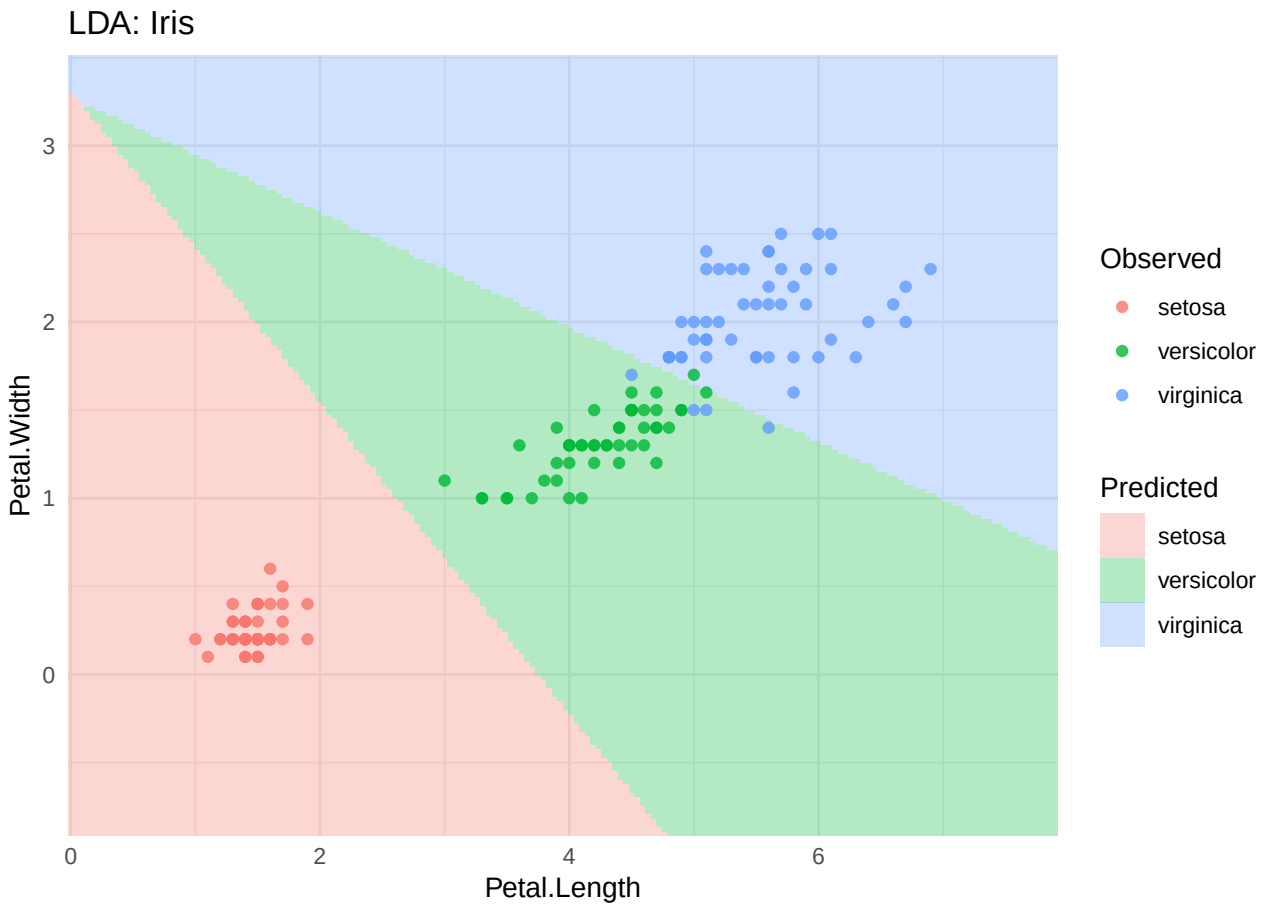
```r
plot_decision_regions(multinom_fit, iris_two,
                      c("Petal.Length", "Petal.Width"), "Species",
                      "Multinomial logistic regression: Iris")
```




### 4.2 Multiclass LDA

```
iris_lda <- lda(Species ~ Petal.Length + Petal.Width, data = iris_two)
iris_lda_class <- list(fit = iris_lda)
class(iris_lda_class) <- "lda_class"

plot_decision_regions(iris_lda_class, iris_two,
                      c("Petal.Length", "Petal.Width"), "Species",
                      "LDA: Iris")
```




### 4.3 Multiclass QDA

```
iris_qda <- qda(Species ~ Petal.Length + Petal.Width, data = iris_two)
iris_qda_class <- list(fit = iris_qda)
class(iris_qda_class) <- "qda_class"

plot_decision_regions(iris_qda_class, iris_two,
                      c("Petal.Length", "Petal.Width"), "Species",
                      "QDA: Iris")
```




## 5. Wrap-up: What should you remember?

| Method | What is modelled? | Covariance assumption | Decision boundary in original features |
|---|---|---|---|
| Binary / multinomial logistic regression | $P(Y = k \mid \mathbf{X})$ directly | No Gaussian covariance assumption | Linear pairwise boundaries |
| LDA | Gaussian $\mathbf{X} \mid Y = k$ | Same covariance across classes | Linear pairwise boundaries |
| QDA | Gaussian $\mathbf{X} \mid Y = k$ | Different covariance by class | Quadratic pairwise boundaries |

**Final comprehension check (TA-led, 3 minutes)**

1. A classification problem has approximately Gaussian predictors within each class, but the classes have very different covariance matrices. Which method would you consider first, and why?
2. A training dataset has only a few observations per class and many predictors. Which may be more stable: LDA or QDA? Why?
3. A logistic regression fit reports very large coefficients and warnings about perfect prediction. What issue should you investigate? Name one potential remedy.
4. A model achieves 95% training accuracy and 60% test accuracy. Which value is most relevant to estimating performance on unseen data, and what might explain the difference?

**Reminder:** A flexible decision boundary does not guarantee better test performance. Evaluate candidate classifiers on held-out data, and consider whether their assumptions are appropriate.